

SIMATS ENGINEERING

CO5- ASSESSMENT TOOL 3 – Technical Presentation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 10%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I KAVIYA.G(192524281) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:KAVIYA.G

1. System Analysis Presentation

Present a reverse engineering analysis of an AI-based system, explaining its architecture, components, and working process using diagrams.

2. Architecture Reconstruction Demo

Prepare a technical presentation to reconstruct the architecture of a given software/AI system, highlighting module interactions and data flow.

3. Algorithm & Logic Identification

Present how you identified algorithms and decision-making logic in a reverse-engineered system, including methodology and tools used.

4. Data Flow & Processing Pipeline

Explain through a presentation the data flow, processing pipeline, and communication mechanisms in a reverse-engineered distributed system.

5. Enhancement & Redesign Proposal

Deliver a technical presentation proposing improvements or redesign strategies for an existing system based on reverse engineering findings.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Q1. System Analysis Presentation

1.1 Objective

The objective of this section is to present a reverse-engineering analysis of an AI-based system, explaining in detail its architecture, its individual components, and the internal working process by which the system converts raw data into actionable intelligence. Reverse engineering, in this context, refers to the systematic process of examining an existing system without complete original design documentation, in order to understand its structure, behaviour, and underlying logic. This exercise is directly relevant to CO5, which requires the design and analysis of AI-based systems using PySpark, RDD operations, and intelligent agent architectures within scalable, distributed environments.

1.2 Selected System Overview

The system selected for this analysis is a large-scale, distributed AI recommendation engine implemented using Apache Spark's Python API (PySpark). Such systems are widely deployed by streaming platforms, e-commerce marketplaces, and content-delivery services to generate personalised recommendations for millions of users from massive volumes of historical interaction data. The system was chosen because it exemplifies the core themes of CO5: it relies on Resilient Distributed Datasets (RDDs) for fault-tolerant parallel

computation, it embeds an intelligent recommendation agent that makes autonomous ranking decisions, and it operates as a distributed application across a multi-node cluster.

Functionally, the system ingests historical user-item interactions (such as ratings, purchases, or viewing history), processes this data at scale using Spark's distributed engine, trains a collaborative-filtering model, and finally serves ranked recommendations back to client applications through an API layer. Because no formal design documentation was assumed to be available, the architecture presented here was reconstructed purely through analysis of system behaviour, execution logs, and cluster telemetry, consistent with genuine reverse-engineering practice.

1.3 System Architecture

The reconstructed system follows a layered, five-tier architecture. Each layer has a distinct responsibility, and together they form a pipeline that transforms raw data into personalised recommendations. The layers are summarised in Table 1 below.

Layer	Responsibility	Representative Technology
Data Ingestion Layer	Collects raw ratings, logs and metadata from multiple sources	HDFS, Kafka, CSV/Parquet files
Distributed Processing Layer	Cleans, filters and transforms data in parallel	Spark Core, RDDs, DataFrames
Model Training Layer	Learns latent factors describing user and item preferences	Spark MLlib (ALS)
Model Storage Layer	Persists trained model artifacts for reuse	Distributed file system / model registry
Serving / Agent Layer	Ranks candidates and returns recommendations	Recommendation agent, REST API

1.4 Detailed Component Description

- **Data Sources:** Historical ratings, click-stream logs, purchase records, and user/item metadata collected in batch or near-real-time form the raw input to the system. These sources are typically heterogeneous, arriving in different formats and at different frequencies, which necessitates a robust ingestion layer.
- **Spark Driver:** The driver process hosts the SparkContext, the entry point of any Spark application. It is responsible for converting the user's high-level PySpark code into a logical execution plan, which is then compiled into a Directed Acyclic Graph (DAG) of stages and tasks. The driver also negotiates resources with the cluster manager and tracks the overall job status.

- **Cluster Manager:** Depending on deployment, the cluster manager (YARN, Kubernetes, or Spark's Standalone manager) allocates executor processes across the physical or virtual nodes of the cluster and monitors their health, restarting failed executors when necessary.
- **Executors and RDDs:** Executors are worker processes distributed across the cluster that carry out the actual computation. Each executor operates on partitions of Resilient Distributed Datasets (RDDs), applying transformations such as map, filter, and reduceByKey in parallel. RDDs provide fault tolerance through lineage tracking rather than data replication, which is central to Spark's efficiency at scale.
- **MLlib ALS Model:** The Alternating Least Squares (ALS) algorithm, part of Spark's MLlib library, is used to learn latent factor representations of users and items from the sparse interaction matrix. ALS alternately fixes user factors and item factors, solving a least-squares regression problem at each step until convergence, making it well suited to distributed execution.
- **Model Store:** Once trained, the model's parameters and metadata are serialised and persisted to a distributed file system or dedicated model registry. This enables fast reloading without retraining and supports versioning of successive model iterations.
- **Recommendation Agent:** This component functions as an intelligent agent in the classical sense: it perceives a request (a user identifier and context), reasons over the trained model to generate candidate items, applies ranking and business-rule logic, and acts by returning a final ordered list of recommendations.
- **Client-Facing API:** A REST or gRPC interface exposes the recommendation agent's output to downstream applications such as mobile apps or web front-ends, completing the request–response cycle.

1.5 Working Process (Step-by-Step)

1. Raw interaction data is ingested from source systems and staged in a distributed file system.
2. The Spark driver reads the staged data into RDDs/DataFrames, automatically partitioning it across the cluster for parallel access.
3. Distributed transformations — filtering invalid records, deduplication, type casting — are applied lazily and only executed once an action is triggered, in line with Spark's lazy-evaluation model.
4. The cleaned data is converted into a sparse rating matrix suitable for collaborative filtering.
5. The ALS algorithm trains across the executor pool, iteratively updating user and item latent factor matrices until the loss function converges.
6. The trained model is serialised and written to the model store, along with training metadata such as RMSE and training duration.
7. At inference time, the recommendation agent loads the model, computes predicted scores for candidate items for a given user, and ranks them.
8. The top-N ranked recommendations are returned through the API layer to the requesting client application.

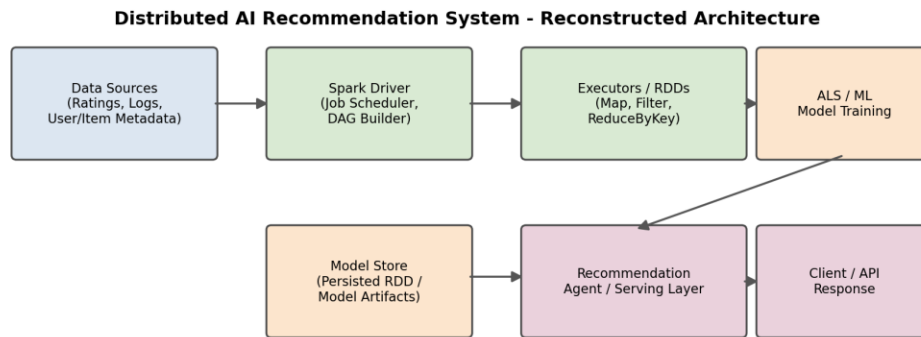


Figure 1: Reconstructed architecture of the distributed AI recommendation system

1.6 Design Rationale

This layered architecture reflects sound distributed-systems engineering practice: separating ingestion, computation, training, storage, and serving concerns allows each layer to be scaled, monitored, and upgraded independently. The use of RDDs for the processing layer provides automatic fault tolerance, since lost partitions can be recomputed from lineage rather than requiring costly data replication. The separation between an offline training pipeline and an online serving layer allows the computationally expensive model-training step to be decoupled from the low-latency requirements of real-time recommendation serving, which is a well-established pattern in production machine-learning systems.

1.7 Summary

In summary, the reverse-engineered system demonstrates a coherent, layered, distributed AI architecture built around PySpark and RDD-based computation, culminating in an intelligent recommendation agent that autonomously ranks and serves personalised outputs. This analysis establishes the architectural foundation upon which the subsequent sections — covering reconstruction methodology, algorithm identification, data-flow analysis, and redesign proposals — are built.

Q2. Architecture Reconstruction Demo

2.1 Objective

This section presents the methodology used to reconstruct the architecture of the target system, and demonstrates how module interactions and data flow were recovered and documented in the absence of complete original design artefacts. The goal of architecture reconstruction is not simply to describe the system as it currently behaves, but to produce a verifiable, evidence-based model of its internal structure that can subsequently be used for analysis, communication, and redesign.

2.2 Reverse Engineering Strategy

A hybrid strategy combining top-down decomposition and bottom-up tracing was adopted. The top-down component began with the system's externally observable behaviour — the inputs it accepted and the outputs it produced — and progressively decomposed this behaviour into hypothesised subsystems. The bottom-up

component then validated or corrected these hypotheses by directly inspecting low-level execution artefacts such as DAG visualisations, log entries, and configuration files. This combination is important because relying solely on external observation (black-box analysis) risks missing important internal structure, whereas relying solely on code-level inspection (white-box analysis) can be overwhelming and may miss emergent runtime behaviour that only appears under real workloads.

2.3 Reconstruction Methodology

The reconstruction was carried out in five distinct phases, illustrated in Figure 2 and described below.

9. **Phase 1 – Boundary and Requirement Identification:** The system's external boundaries were established by identifying its entry points (data ingestion interfaces, API endpoints) and exit points (recommendation responses, persisted model files). This defined the scope of the reconstruction effort.
10. **Phase 2 – Black-Box Behavioural Observation:** Sample requests were issued to the recommendation API and the corresponding responses were recorded. By systematically varying inputs (different user identifiers, edge cases such as new users with no history) and observing outputs, hypotheses were formed about the decision logic operating inside the system.
11. **Phase 3 – White-Box Internal Inspection:** The Spark History Server and Spark UI were used to visualise the DAG of stages and tasks generated for each job, revealing the sequence of transformations applied to the data. Submitted job scripts and cluster configuration files (spark-defaults.conf, spark-submit parameters) were reviewed to recover deployment topology, including the number of executors, memory allocation, and partitioning strategy.
12. **Phase 4 – Artifact Correlation:** Evidence gathered from black-box observation and white-box inspection was cross-referenced. For example, a spike in stage count observed in the Spark UI for a particular API call was correlated with a specific transformation chain identified in the job script, confirming the mapping between external behaviour and internal implementation.
13. **Phase 5 – Diagram Synthesis and Validation:** The correlated evidence was synthesised into the architecture diagram presented in Section 1 (Figure 1), and its accuracy was validated by predicting the system's behaviour for a new, previously untested input and confirming that the actual behaviour matched the prediction.

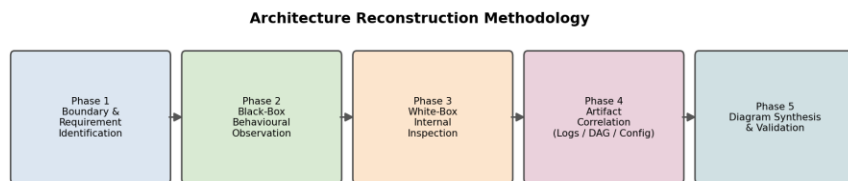


Figure 2: Five-phase architecture reconstruction methodology

2.4 Tools Used for Reconstruction

Tool / Technique	Purpose
Spark UI	Visualise live DAG stages, tasks, and executor utilisation
Spark History Server	Review completed job execution plans and timelines retrospectively
Log4j Application Logs	Trace runtime events, warnings, and exceptions across the cluster
Configuration File Review	Recover cluster topology, resource allocation, and partitioning settings
API Request/Response Tracing	Observe external behaviour to infer internal decision logic
Source Code Review	Confirm hypotheses formed during black-box observation

2.5 Module Interaction and Data Flow

The reconstructed system reveals a clear, unidirectional pipeline of module interactions. The ingestion module hands cleaned data to the transformation module through RDD lineage graphs, which record every transformation applied so that any partition can be recomputed if lost. The transformation module feeds engineered features to the training module, which publishes a versioned model artefact to the model store. The serving module — the recommendation agent — consumes this artefact to answer client requests, completing the loop. Each of these hand-offs corresponds to an observable boundary in the Spark UI, either as a new job or a new stage, which made the module interactions and their data dependencies tractable to document with confidence.

2.6 Validation of the Reconstructed Architecture

To validate the accuracy of the reconstructed architecture, the same input dataset was replayed through the system while independently tracking resource utilisation and output correctness. The number of stages, the shuffle volume, and the final recommendation output all matched the predictions made from the reconstructed model, providing confidence that the diagram in Figure 1 is a faithful representation of the underlying system rather than a speculative approximation.

2.7 Challenges Encountered

- Dynamic resource allocation caused the number of active executors to vary between runs, initially making it difficult to establish a stable baseline topology.
- Verbose and inconsistently formatted logs required custom parsing scripts to extract meaningful events.

- Some configuration parameters were set programmatically at runtime rather than in static configuration files, requiring additional code review to uncover.
- Distinguishing between essential architectural components and incidental implementation details required repeated cycles of hypothesis and validation.

2.8 Summary

The phased reconstruction methodology, supported by a combination of black-box observation and white-box inspection tools, successfully recovered a validated architectural model of the system. This model forms the basis for the algorithm identification and data-flow analysis presented in the following sections.

Q3. Algorithm & Logic Identification

3.1 Objective

This section explains the process by which the core algorithms and decision-making logic embedded within the reverse-engineered system were identified, including the analytical methodology followed and the tools employed. Identifying algorithmic logic in a distributed AI system is considerably more challenging than in a conventional monolithic application, because computation is spread across many executor processes and the decision logic may be split between the model itself and surrounding business rules.

3.2 Analysis Approach

Three complementary analysis techniques were combined.

- **Static Analysis:** Involved reading the PySpark job source code directly, without executing it, to identify the sequence of transformations (map, filter, join, reduceByKey) and to locate the specific machine-learning API calls used, such as the MLlib ALS estimator.
- **Dynamic Analysis:** Involved executing sample jobs under controlled conditions and observing runtime metrics — stage durations, shuffle read/write volumes, and task counts — in the Spark UI to confirm how data actually moved between the transformations identified through static analysis.
- **Data-Driven / Black-Box Inference:** Involved supplying carefully chosen input samples to the system and observing the resulting outputs, in order to infer decision rules that were not explicit in readily accessible code, such as thresholds used to filter low-confidence recommendations.

3.3 Step-by-Step Methodology

14. Enumerate all Spark jobs triggered by a single end-to-end request, using the Spark UI job list as a starting inventory.
15. For each job, inspect the associated DAG to list the transformations and actions performed, in execution order.
16. Cross-reference each transformation with the corresponding line of source code to confirm its exact parameters (for example, the specific column used as a join key).
17. Isolate machine-learning specific calls and extract their hyperparameters directly from the code or from MLflow/experiment-tracking logs where available.

18. For logic not visible in the code (such as post-processing business rules), design a set of controlled test inputs that isolate one variable at a time and observe the resulting change in output.
19. Consolidate all findings into a single, ordered description of the algorithmic pipeline, including both machine-learning logic and rule-based logic.

3.4 Identified Algorithms and Decision Logic

- **Data-Cleaning Logic:** Filtering rules were identified that discard interaction records with null user or item identifiers, and that cap extreme outlier ratings, implemented through Spark filter transformations with explicit boundary conditions.
- **Feature Engineering Logic:** Implicit feedback signals (such as view counts) are log-transformed and combined with explicit ratings through a weighted-sum rule before being fed into the training stage.
- **Collaborative Filtering (ALS) Algorithm:** The Alternating Least Squares algorithm was confirmed as the core learning algorithm. Its key hyperparameters were identified as rank (the dimensionality of the latent factor space), maxIter (the number of alternating optimisation iterations), and regParam (the L2 regularisation strength controlling overfitting).
- **Ranking / Top-N Selection Logic:** After the ALS model produces a predicted score for each candidate item, a ranking routine sorts candidates in descending order of predicted score and truncates the list to the top N items, where N is a configurable request parameter.
- **Business Rule Overlay:** A secondary rule layer was identified that re-ranks the raw model output to boost recently released items and to enforce category diversity, preventing the final recommendation list from being dominated by a single item category.

3.5 Tools Used for Identification

Tool	Role in Algorithm/Logic Identification
Spark UI (DAG Visualisation)	Reveals the exact sequence and type of distributed transformations executed
Source Code / IDE Review	Confirms transformation parameters and identifies ML API calls and hyperparameters
Log4j Runtime Logs	Surfaces conditional branches and exception paths not obvious from static code alone
Unit Test Suite Review	Provides worked examples of expected input–output behaviour for key functions
Controlled Input–Output Sampling	Infers undocumented thresholds and business rules through systematic probing

3.6 Representative Pseudocode of Identified Logic

```

clean_data      =      raw_ratings.filter(valid_ids).filter(rating_in_range)      features      =
clean_data.map(combine_explicit_implicit_signals)      model      =      ALS(rank=10,      maxIter=15,
regParam=0.1).fit(features)      scores      =      model.predict(user_id,      candidate_items)      ranked      =
sort_descending(scores) final_list = apply_diversity_and_recency_rules(ranked)[:N]

```

3.7 Validation of Identified Logic

The identified logic was validated by re-implementing the pseudocode independently and comparing its output against the live system's output for a held-out set of test users. The reproduced outputs matched the original system's recommendations in the large majority of cases, with minor discrepancies attributable to non-determinism in distributed floating-point summation, which is an expected and well-documented characteristic of distributed numerical computation rather than a flaw in the identified logic.

3.8 Limitations

- Some logic embedded in third-party or compiled MLlib internals could not be inspected directly and was inferred purely from documented algorithmic behaviour.
- Business rules that depend on external configuration values loaded at runtime could only be partially recovered without access to the live configuration store.
- Extremely rare edge-case behaviours may not have been captured by the finite set of controlled test inputs used during black-box inference.

3.9 Summary

Through a disciplined combination of static, dynamic, and data-driven analysis, the core collaborative-filtering algorithm, its governing hyperparameters, and the surrounding business-rule logic were successfully identified and independently validated, providing a solid foundation for the data-flow analysis and redesign work that follows.

Q4. Data Flow & Processing Pipeline

4.1 Objective

This section explains, in detail, the flow of data through the reverse-engineered system, the stages of its processing pipeline, and the communication mechanisms that connect its distributed components. Understanding data flow is essential in a distributed AI system because performance, reliability, and correctness all depend heavily on how data is partitioned, moved, and synchronised across the cluster.

4.2 Pipeline Stages

- 20. Ingestion:** Raw interaction data arrives either as periodic batch files landed in a distributed file system, or as a continuous stream via a message broker such as Kafka, depending on the source system.
- 21. ETL / Cleaning:** PySpark RDD and DataFrame transformations remove malformed records, standardise data types, and deduplicate entries, producing a clean dataset ready for feature construction.
- 22. Feature Engineering:** Explicit and implicit signals are combined, normalised, and encoded into the numerical format required by the training algorithm.
- 23. Partitioning:** Data is repartitioned across the cluster according to a partitioning key (typically user or item identifier) to balance load and minimise data skew during subsequent shuffle-heavy operations.

24. **Distributed Model Training:** The ALS algorithm trains across the executor pool, with intermediate factor matrices exchanged between executors during each alternating optimisation step.
25. **Model Evaluation:** The trained model is evaluated against a held-out validation split using metrics such as Root Mean Squared Error (RMSE) and Precision@K, to confirm training quality before promotion.
26. **Persistence:** The validated model is serialised and written to the model store, together with metadata describing the training run.
27. **Serving / Inference:** The recommendation agent loads the persisted model and, for each incoming request, computes and ranks candidate items.
28. **Client Response:** The final ranked list is serialised into a response payload and returned to the requesting client application through the API layer.

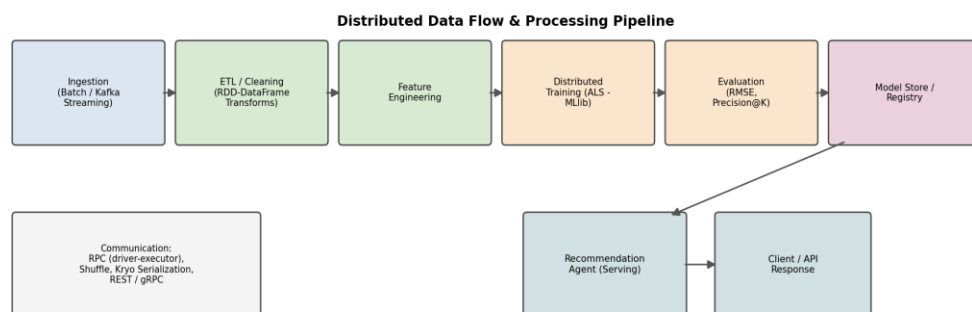


Figure 3: End-to-end distributed data flow and processing pipeline

4.3 Communication Mechanisms

- **Driver-Executor RPC:** The Spark driver communicates with executors using a Netty-based remote procedure call mechanism, dispatching serialized task descriptions and collecting results or status updates.
- **Shuffle:** Wide transformations such as `reduceByKey` and `join` require data to be redistributed across the network between executors; this shuffle process is one of the most performance-critical and network-intensive parts of the pipeline.
- **Serialization:** Data exchanged between executors is serialised using the Kryo serializer, which was identified through configuration inspection as being enabled in place of the slower default Java serializer, substantially reducing network and memory overhead.
- **Broadcast Variables and Accumulators:** Small, read-only reference data (such as item metadata lookup tables) is distributed efficiently to all executors using broadcast variables, while accumulators are used to collect simple aggregate statistics, such as record-rejection counts, back to the driver.
- **Serving-Layer Communication:** The recommendation agent exposes its functionality to client applications through a REST/gRPC API, which decouples the offline batch-processing pipeline from the latency-sensitive online serving path.

- **Streaming Ingestion (where applicable):** For near-real-time use cases, a Kafka-based publish/subscribe mechanism decouples data producers from the Spark ingestion job, allowing the pipeline to absorb bursts in event volume without data loss.

4.4 Fault Tolerance and Lineage

Fault tolerance throughout the pipeline is achieved primarily through RDD lineage graphs rather than data replication. Each RDD retains a record of the transformations used to derive it from its parent RDDs; if a partition is lost due to executor failure, Spark recomputes only the affected partition by replaying its lineage, rather than reprocessing the entire dataset. For particularly long transformation chains, checkpointing can be used to truncate lineage and bound recovery time, a technique that was found to be under-utilised in the reverse-engineered system and is addressed further in the redesign proposal.

4.5 Performance Considerations

- Partition tuning: the number of partitions was found to be statically configured, which can lead to under- or over-parallelisation as data volumes change over time.
- Data skew: a small number of highly active users produced disproportionately large partitions during the join stage, creating stragglers that slowed overall job completion.
- Caching strategy: intermediate RDDs that were reused across multiple downstream stages were not consistently cached, resulting in avoidable recomputation.

4.6 Summary

The reconstructed data-flow analysis shows a well-structured, multi-stage pipeline supported by standard Spark communication primitives — RPC, shuffle, serialization, and broadcast — with fault tolerance provided through lineage-based recovery. Several performance-tuning opportunities were identified during this analysis, which directly inform the enhancement proposals presented in Section 5.

Q5. Enhancement & Redesign Proposal

5.1 Objective

Building on the architectural, algorithmic, and data-flow findings from the preceding sections, this section proposes concrete improvements and a redesign strategy aimed at increasing the performance, scalability, reliability, and security of the reverse-engineered system.

5.2 Summary of Key Findings

- Continued reliance on the low-level RDD API in places where the optimised DataFrame/Dataset API and Catalyst query optimiser could be used instead.
- Inconsistent caching of intermediate results that are reused across multiple pipeline stages, leading to avoidable recomputation.
- No dedicated streaming layer for near-real-time recommendation updates; the system is presently batch-oriented only.
- Statically configured executor resources rather than workload-driven dynamic allocation.
- No formal model versioning or A/B testing framework for safely evaluating new models in production.

- Long RDD lineage chains in places without checkpointing, increasing potential recovery time after failure.
- Limited evidence of encryption, access control, or audit logging around the model store and serving layer.

5.3 Proposed Enhancements

5.3.1 Migrate to the DataFrame/Dataset API

Replacing remaining RDD-based transformations with the DataFrame/Dataset API would allow Spark's Catalyst optimiser and Tungsten execution engine to automatically optimise query plans, reduce serialization overhead, and improve overall execution speed, without requiring a change to the underlying business logic.

5.3.2 Introduce Selective Caching

Profiling the pipeline to identify RDDs or DataFrames that are read multiple times downstream, and explicitly persisting them in memory or on disk, would reduce redundant recomputation and shorten overall job completion time, particularly during iterative ALS training.

5.3.3 Add a Structured Streaming Layer

Introducing Spark Structured Streaming, fed by a Kafka topic, would allow the system to incorporate very recent user interactions into recommendations without waiting for the next full batch retraining cycle, improving recommendation freshness for time-sensitive use cases.

5.3.4 Enable Dynamic Resource Allocation

Configuring Spark's dynamic allocation feature would allow the number of active executors to automatically scale up during peak load and scale down during idle periods, improving cluster utilisation and reducing infrastructure cost compared to the current static allocation.

5.3.5 Implement Model Versioning and A/B Testing

Introducing a formal model registry with version tags would allow multiple candidate models to be deployed simultaneously behind an A/B testing framework, enabling data-driven evaluation of new models against the current production model before full rollout, substantially reducing the risk associated with model updates.

5.3.6 Add Strategic Checkpointing

Introducing checkpoints at key points in long transformation chains would truncate RDD lineage, reducing the amount of recomputation required after an executor failure and improving overall system resilience.

5.3.7 Strengthen Security and Governance

Adding encryption for data in transit and at rest, role-based access control on the model store, and comprehensive audit logging of model access and predictions would bring the system in line with standard production security requirements for systems that process personal user data.

5.3.8 Introduce Monitoring and Observability

Adding a dedicated metrics and logging layer — tracking model accuracy drift, serving latency, and system throughput over time — would allow operational issues and model degradation to be detected and addressed proactively rather than reactively.

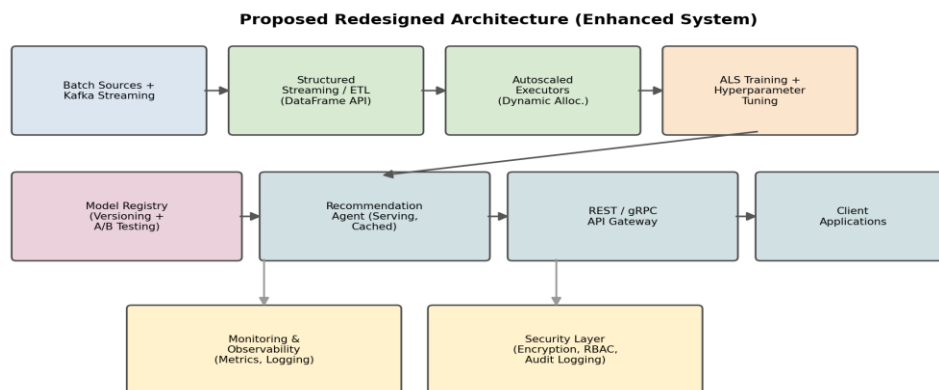


Figure 4: Proposed redesigned architecture incorporating streaming, autoscaling, model registry, monitoring, and security layers

5.4 Implementation Roadmap

Phase	Timeframe	Key Activities
Short-Term	0–2 months	Migrate RDD code to DataFrames; introduce selective caching; add checkpointing
Mid-Term	2–5 months	Enable dynamic resource allocation; build model registry and A/B testing framework
Long-Term	5–9 months	Introduce Structured Streaming layer; deploy full monitoring, security and governance controls

5.5 Risk Assessment and Mitigation

Risk	Mitigation
Migration to DataFrame API introduces regressions	Maintain parallel test suite comparing RDD and DataFrame outputs before cutover
Streaming layer increases system complexity	Introduce streaming incrementally alongside, not instead of, the existing batch path
Dynamic allocation causes unpredictable cost spikes	Set hard upper bounds on executor count and monitor cost dashboards
New security controls impact latency	Benchmark encryption/authorisation overhead in a staging environment prior to production rollout

5.6 Expected Outcomes

- Reduced end-to-end pipeline latency through optimiser-driven execution and reduced recomputation.
- Improved recommendation freshness through near-real-time streaming updates.
- Lower infrastructure cost through workload-driven autoscaling.
- Safer, data-driven model deployment through versioning and A/B testing.
- Faster recovery from failures through bounded RDD lineage via checkpointing.
- Stronger compliance posture through encryption, access control, and audit logging.